応用情報技術者試験　テクノロジ系　#9

データベース

用語集まとめ ― 過去問10回分（令和3〜7年度）とシラバスの用語例から



全体像 シラバスの5つの項目を、そのまま5章で読む

1 データベース方式
関係・階層・網・NoSQL（キーバリュー・ドキュメント・列指向・グラフ）・3層スキーマ

2 データベース設計
E-R図・UMLの多重度・キー・制約・CASCADE・正規化・容量見積り

3 データ操作
和・差・積・直積・選択・射影・結合・SQL（GROUP BY・副問合せ・外部結合・ウィンドウ関数・ビュー）

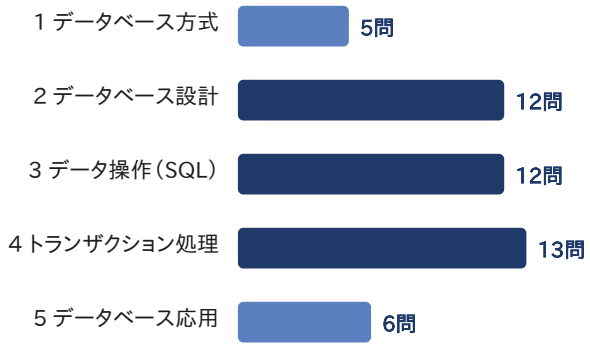
4 トランザクション処理
ACID・ロック・MVCC・2相コミット・障害回復・チェックポイント・インデックス

5 データベース応用
データウェアハウス・データレイク・スタースキーマ・OLAP・分散の透過性・CAP・BASE

午前での出方	
問題数	毎回 **4〜5問**（問25〜問31あたり）。10回で48問
同じ問題の再出題	1組（ハッシュインデックス：R5秋・R7秋）。似た問題は多い（CASCADE、チェックポイント）
問題の型	**表やSQLを読んで答えを数える問題**と、**用語の説明の入れ替え**が半々

この資料の読み方：SQLの問題は表を1行ずつ追えば必ず解ける。用語の問題は「どの説明がどの用語のものか」を表で覚える。

設計・SQL・トランザクションの3つで、出題の8割



令和3年度春期〜令和7年度秋期の48問を、章ごとに数えた（同じ問題も1問ずつ数えた）

よく出る型	中身	過去問
多重度を読む	条件文や図から 1・*・1..* を決める	R3春29・R5春29・R6秋29
正規形を判定	主キーの一部で決まる項目→第2正規形でない	R3秋28・R4春28・R7春26
SQLの結果を数える	平均との比較、NOT EXISTS、COALESCE	R4秋28・R5秋29・R6春26
集合演算	和・差・積の結果の表	R3秋26・R4秋27・R7春28
障害回復	チェックポイントの後にコミット→ロールフォワード	R4秋29・R5秋30・R6春27
NoSQL・CAP	4つの型、AとPなら整合性を捨てる	R3春28・R4春26・R7秋26

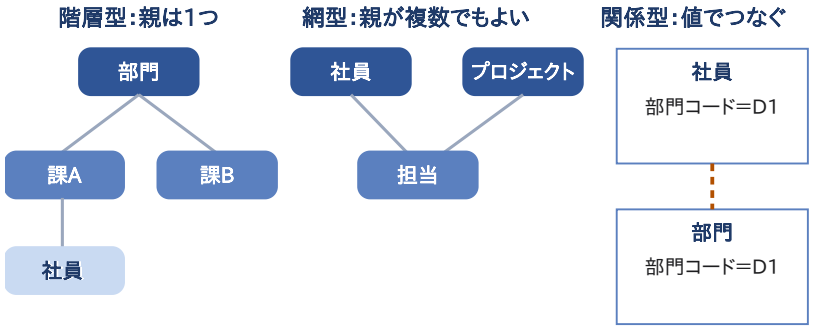
第1章

データベース方式

関係・階層・網・オブジェクト指向 ／ NoSQLの4つの型 ／ 3層スキーマとデータ独立性 ／ 関係モデルの用語 ／ DBMSの機能と製品



関係データベースは「値」で表をつなぐ。階層型・網型は道筋で



階層型＋網型＝構造型データベース（ポインタでたどる）

種類	特徴
関係データベース	2次元の表。共通の列の値で関連付け。SQL
階層型（HDB）	木構造。子から見た親は1つ。多対多は不可
網型（NDB）	子が複数の親をもてる。バックマン線図で表す
オブジェクト指向（OODB）	データと操作（メソッド）をまとめて記録。継承
オブジェクト関係	関係DBに型の継承や操作の定義を足した（R6春29ア）
XMLデータベース	XML文書をタグの階層のまま記録。XQuery
インメモリ	データを主記憶に置いて高速に処理

NoSQLは4つの型。選択肢の説明から型を言い当てる

キーバリュー型

user:1001 → 値（型は自由）
キー1つに値1つ
Redis

ドキュメント指向

{{ 名前:…, 住所:{{…}} }}
JSONの階層ごと1件に
MongoDB・CouchDB

列指向（ワイドカラム）

行ごとに列の数が違ってよい
列に複数の値（R7春24）
Apache Cassandra

グラフ型

ノード ― リレーション
どちらもプロパティをもつ
Neo4j（R3春28）

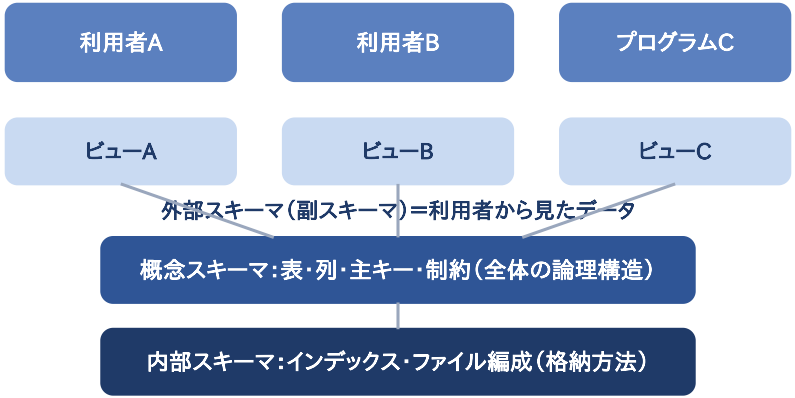
NoSQLは「関係データベース以外」の総称。表の形にとらわれず、多数のサーバに分散して大量のデータを扱う（ビッグデータの基盤、R6春29）。

R5春 問26：項目数も内容も予測できない商品データ（JSON）の格納

	格納方法	判定
ア	階層型で、項目名を上位・値を下位の2階層に	木の形を前もって決める方式
イ	グラフDBで、項目名のノードと値のノード	どの値がどの項目か分からない
ウ	**ドキュメントDBで、項目構成の違いを区別せず1件ごとに**	**正解**
エ	関係DBで、各項目名を個別の列名に	新しい項目のたびに表を変更

「行ごとに任意の列数＋列ごとに複数の値」はワイドカラム（R7春24）。「ドキュメント」と迷わせる。

3層スキーマ：見せ方・論理の形・格納方法を分けて、互いに独立に



データ独立性：内部スキーマを変えても概念スキーマは変わらない（物理的独立性）。概念スキーマを変えてもビューでプログラムを守れる（論理的独立性）。

R4春 問27：内部スキーマの設計に含まれるもの

	設計の作業	どのスキーマか
ア	**SQLの応答時間を良くするインデックスの定義**	**内部（正解）**
イ	1対多・多対多を明示するE-Rモデルの作成	概念
ウ	一意性制約・参照制約の設定	概念
エ	冗長性を排除する正規化	概念

データモデルも3段階：**概念**データモデル（E-R図・UML、DBMSに依存しない）→ **論理**データモデル（表・キーの形）→ **物理**データモデル（型・索引・分割）。

関係モデルの言葉：行はタプル、列は属性、値の範囲は定義域

関係スキーマ：社員（社員番号, 氏名, 部門コード）

	社員番号	氏名	部門コード
タプル（行）	1001	情報 太郎	D1
	1002	試験 花子	D2

実現値＝今入っている行の集まり　　属性（列）

定義域（ドメイン）＝属性がとりうる値の集合（部門コードなら D1〜D9）

DBMSの機能	中身
データベース定義機能	表・ビュー・索引を定義（DDL）
データベース操作機能	検索・追加・更新・削除（DML）
データベース制御機能	同時実行制御・障害回復・機密保護（DCL）
保全機能	制約で誤ったデータを入れさせない
データ機密保護機能	利用者ごとのアクセス権（GRANT）

関係DBの製品	NoSQLの製品
MySQL・PostgreSQL・SQLite	MongoDB・CouchDB（ドキュメント）、Redis（キーバリュー）、Cassandra（列指向）、Neo4j（グラフ）

> 関係（表）は、行の重複がなく、行の順番・列の順番に意味がない。この点が表計算の表と違う。

第2章

データベース設計

データ項目の標準化 ／ E-R図とUMLの多重度 ／ キーの種類 ／ 制約とCASCADE ／ 関数従属と正規化 ／ 容量見積り



🔗 多重度は「相手1つから見て、こちらがいくつか」を線の端に書く



何度も注文する　　一度に1つ以上の商品　　明細1行に商品は1種類

R3春 問29：a＝顧客, b＝注文, c＝注文明細, d＝商品（ア）

表記	意味	条件文の言い方
1	ちょうど1つ	「いずれか一つの」
0..1	0か1	「もつこともある」
1..*	1つ以上	「1人以上が」「必ず」
（0..）	0個以上	「何度も」「いなくてもよい」

多対多の関連は、間に連関エンティティ（注文明細・所属履歴）を置いて、1対多の2本に分ける。

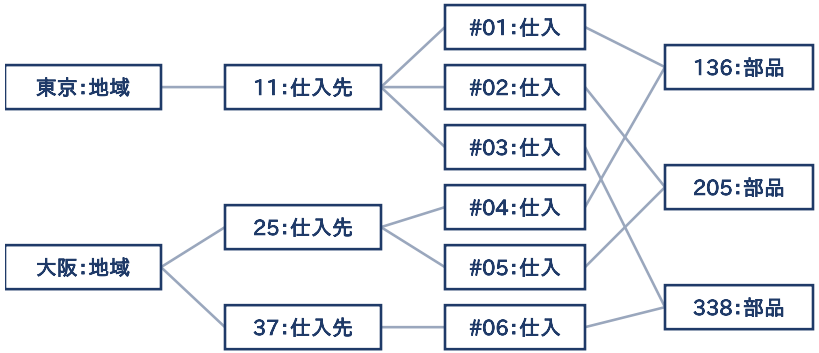
R5春 問29：部門 1 ― a 所属履歴 b ― 1 社員

条件	読み取り
(1) 部門には1人以上の社員が所属	部門から見て履歴は1件以上 → a＝1..*
(2) 社員はいずれか一つの部門に所属	社員から見て履歴は1件以上 → b＝1..*
(3) 所属の履歴を記録	異動すれば履歴が増える（上限なし）

答えはエ（1..*, 1..*）。0..* にすると「社員のいない部門」「部門のない社員」を許してしまう。

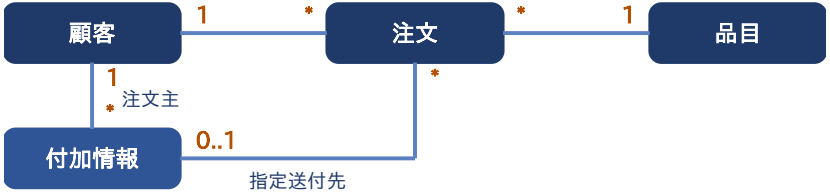
オブジェクト図は線を数える。外部キーは必ず「多」の側に置く

R6秋 問29：オブジェクト図 → 概念データモデル



大阪に仕入先2つ → 地域 1—* 仕入先
仕入先11に仕入3つ → 仕入先 1—* 仕入
仕入は部品1つ、部品に仕入複数 → 仕入 *—1 部品（イ）

R7春 問25：UMLのモデルを関係データベースに実装する



	解釈	判定
ア	どの付加情報の行でも選べる	注文主の一致が制約
イ	付加情報と顧客の行数が一致	1対多なので不要
ウ	**付加情報に顧客への参照制約**	**正解（多の側が参照）**
エ	付加情報に注文への参照制約	向きが逆（注文が参照）

🔑 キーは包含関係で覚える。外部キーだけは「他の表を参照する」もの



外部キー：他の表の候補キー（ふつうは主キー）を参照する列。参照先にない値は入れられない（参照制約）。**サロゲートキー**：意味のない連番を主キーとして足したもの。

R5秋 問27：外部キーの説明		
	説明	**正体**
ア	**ある関係の候補キーを参照する属性（の組）**	**外部キー**
イ	主キー以外で、タプルを一意に識別できる	代替キー
ウ	一意に識別できる属性の組のうち極小	候補キー
エ	一意に識別できる属性の組を含む集合	スーパーキー

制約	SQL	守ること
一意性制約	UNIQUE	重複しない（NULLは可のことも）
主キー制約	PRIMARY KEY	重複なし+NULL不可
参照制約	FOREIGN KEY … REFERENCES	参照先に値がある
検査制約・非NULL制約	CHECK・NOT NULL	条件を満たす・値がある

「親を消したら子も自動で消す」は ON DELETE CASCADE



R5春 問30・R7春 問27はどちらも答えがCASCADE（ア）。誤答にはRESTRICT・SET NULL・SET DEFAULT のほか、INTERSECT・UNIQUE のような別の種類の語句も並ぶ。

R4秋 問26：命名規約だけでは回避できない事象

(1) 末尾に区分語（名・コード・金額・年月日）＋区分語ごとのデータ型
(2) 項目名と意味の辞書で、異音同義語・同音異義語を防ぐ

	事象	防ぐ規約
ア	受信年月日に日付型と文字列型が混在	(1)の型
イ	**受注金額の値の範囲がテーブルで違う**	**どちらも決めていない（正解）**
ウ	賞与金額とボーナス金額（異音同義語）	(2)の辞書
エ	取引先がコードか名か分からない	(1)の区分語

🧩 正規化は「何で決まるか」を見て、決める側を主キーにした表へ分ける

第1正規形：受注（受注番号, 項番, 受注日, 得意先, 商品コード, 数量, 単価）

受注番号	**項番**	受注日	得意先	商品	数量	単価

受注日・得意先は受注番号だけで決まる（部分関数従属）→ 分ける

第2正規形

受注番号	受注日	得意先

受注番号	**項番**	商品	数量	単価

単価は商品コードで決まる（推移関数従属：主キー→商品→単価）→ 分ける

第3正規形

受注番号	受注日	得意先

受注番号	**項番**	商品	数量

商品	単価

正規形	条件（なくすもの）
第1	繰返し項目がない（R4春28の c）
第2	主キーの一部への従属がない（a：部分関数従属）
第3	主キー以外どうしの従属がない（b：推移関数従属）
BCNF	全部の決定項がスーパーキー

R3秋 問28は左の第1正規形の表そのもの。答えは「第1正規形であるが第2正規形でない」（イ）。

R7春 問26：2NF→3NFは「候補キー以外の属性間の関数従属を分解」（イ）。「候補キーの一部から」は1NF→2NF、「候補キー以外から候補キーの一部へ」は3NF→BCNF。

表の設計を問う問題は「その情報を結び付ける列があるか」で解く

R6秋 問28：3つの表だけでは得られない情報

表	列
顧客	顧客ID・氏名・**性別**・生年月日
販売代理店別日別販売	販売代理店ID・**日付**・商品ID・販売数量
商品購入	**顧客ID**・販売代理店ID・商品ID・購入数量

	情報	判定
ア	商品ごとの販売数量の日別差異	日別販売で求まる
イ	性別ごとの売れ筋商品	顧客＋商品購入で求まる
ウ	**代理店ごとの購入者数の日別差異**	**顧客と日付を結ぶ表がない**
エ	代理店ごとの売れ筋商品	日別販売で求まる

容量の見積り（o04）：200バイト×10万件、1ブロック4,000バイト、使用率80%

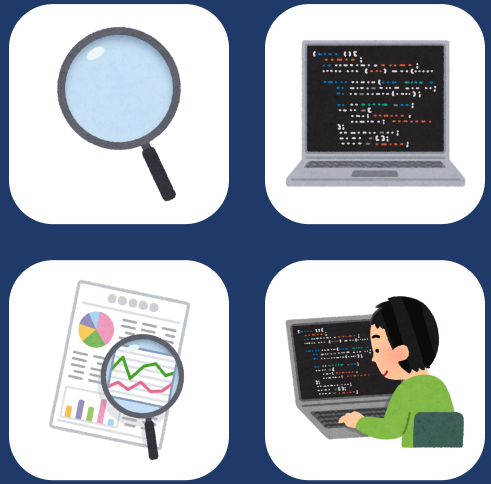
使える領域 4,000 × 0.8 ＝ 3,200バイト
1ブロックに 3,200 ÷ 200 ＝ 16件（端数は切り捨て）
100,000 ÷ 16 ＝ 6,250ブロック（端数は切り上げ）

物理設計の用語	中身
非正規化	速さのため、あえて結合した形で持つ
副次索引	主キー以外の列の索引。作りすぎると更新が遅い
表の分割	行の範囲（水平）や列（垂直）で分けて格納
再編成	散らばった格納を整理し直す

第3章

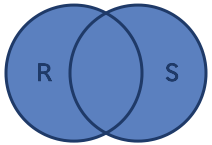
データ操作（SQL）

和・差・積・直積 ／ 選択・射影・結合 ／ GROUP BYとHAVING ／ 副問合せとEXISTS ／ 外部結合とCOALESCE ／ ウィンドウ関数 ／ ビュー ／ ストアドプロシージャ

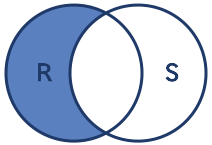


集合演算は行を足し引きする。結合は列を横につなぐ

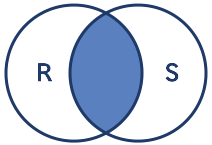
和 R∪S（UNION）



差 R−S（EXCEPT）



積 R∩S（INTERSECT）



過去問	読み取り	答え
R3秋 問26	RとSの行が全部入り、重複は1行 → 和	エ 和
R4秋 問27	従業員−異動＝従業員にだけある3行	イ
R7春 問28	R−(R−S)＝Rから「Rだけ」を除く＝R∩S	イ

> 差は向きで結果が変わる。R4秋27では「異動−従業員」（D080の1行）も選択肢に並ぶ。

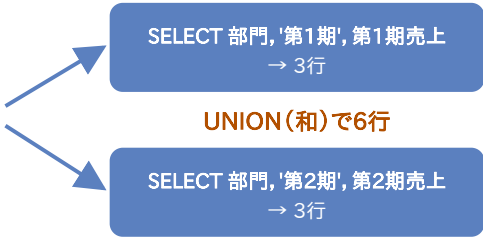
演算	何をするか	SQL
選択	条件に合う**行**を取り出す	WHERE
射影	指定した**列**を取り出す	SELECT 列名
結合	共通の列の値でつなぐ	JOIN … ON
直積	全部の組合せ（行数は掛け算）	CROSS JOIN
商	「すべての〜と組になる」値	（NOT EXISTSの二重）

> 和・差・積ができるのは、列の数と型がそろった「和両立」な表どうしだけ。

横に並んだ列を縦に並べるには、SELECTを2つ作ってUNION

R3秋 問29：部門別売上（部門コード, 第1期売上, 第2期売上）を縦持ちに

部門	第1期	第2期
D01	1,000	4,000
D02	2,000	5,000
D03	3,000	8,000



選択肢	結果
ア INTERSECT	期の値が違うので共通の行はなく0行
イ UNION	**6行（正解）**
ウ CROSS JOIN	3×3=9行で、第1期の列しか出ない
エ INNER JOIN	3行で、第1期の列しか出ない

順	句	役割
1	FROM・JOIN	表を決めてつなぐ
2	WHERE	1行ずつ条件で絞る
3	GROUP BY	同じ値の行をグループに
4	HAVING	グループを条件で絞る
5	SELECT	列・集約関数を求める
6	ORDER BY	並べ替え

> o05：「合計が100万円以上の店舗」は HAVING SUM(売上金額) >= 1000000。WHEREにSUMは書けない。

SQLの結果を数える問題は、副問合せの値を先に計算する

R4秋 問28：利益（販売−仕入）が平均より大きい商品の仕入先コード（DISTINCT）

商品	利益	仕入先	平均360超
A001	200	S1	
B002	200	S2	
C003	100	S2	
D004	900	S1	○
E005	400	S1	○
F006	200	S3	
G007	300	S3	
H008	500	S4	○
I009	500	S5	○
J010	300	S6	

平均 3,600 ÷ 10 = 360 → 4行 → DISTINCT で S1・S4・S5 = 3（ウ）

R5秋 問29：NOT EXISTS（在庫数 > 30 の在庫）

製品	在庫数	30超えがある？	結果
AB1805	20・150	ある	除く
CC5001	200・40	ある	除く
MZ1000	（行なし）	ない	**残る**
XZ3000	30・20	ない（30は超えない）	**残る**
ZZ9900	130・10	ある	除く

残るのは2行（イ）

相関副問合せは、外側の1行（製品）ごとに中のSELECTを実行する。「1つでもあれば除く」がNOT EXISTS。

🔗 在庫のない部品も残すなら外部結合。NULLはCOALESCEで0に

R6春 問26：CASE WHEN 部品.発注点 > a THEN '必要' ELSE '不要'				
部品	発注点	在庫（LEFT OUTER JOIN）	SUM	結果
P01	100	90・90	180	不要
P02	150	150	150	不要
P03	100	NULL（行なし）	NULL→**0**	**必要**

a ＝ COALESCE(SUM(在庫.在庫数), 0)（ウ）

> MINだとP01が90で「必要」になる。COALESCE(…, NULL) では P03 が NULL のままで、比較が真にならず「不要」になる。

結合	残る行
INNER JOIN	両方に対応がある行だけ
LEFT OUTER JOIN	左の表は全部。右がなければNULL
RIGHT OUTER JOIN	右の表は全部
FULL OUTER JOIN	左右とも全部
CROSS JOIN	全部の組合せ（直積）

NULLの扱い	中身
比較	「= NULL」も「> NULL」も真にならない → IS NULL で調べる
集約関数	SUM・AVG・MAX・MINはNULLを無視。行が0ならNULL
COUNT	COUNT(*)は全行、COUNT(列)はNULL以外

ウィンドウ関数は、行を減らさずにグループごとの番号や合計を付ける

R6秋 問30：学生ごとの初回（実施回が最小）の行を取り出す

ROW_NUMBER() OVER (PARTITION BY 学生番号 ORDER BY 実施回) AS 番号
… WHERE 番号 = 1（イ）

学生	実施回	得点	番号
S01	1	70	1
S01	7	80	2
S02	2	85	1
S02	5	82	2
S03	3	83	1
S03	9・12	78・90	2・3
S04	6	100	1

a の候補	番号1になる行
ア ORDER BY 学生番号, 実施回	区切りがなく、表全体の先頭1行だけ
イ PARTITION BY 学生番号 ORDER BY 実施回	**学生ごとの初回（正解）**
ウ … ORDER BY 得点 ASC	学生ごとの最低点
エ … ORDER BY 得点 DESC	学生ごとの最高点

PARTITION BY＝どこで区切るか（GROUP BYに似るが行は減らない）、ORDER BY＝区切りの中の並べ方。SQL文1の「MIN(実施回)で初回を求めて結合」と同じ結果になる。

🏷️ ビューを更新できるのは、1行が基の表の1行に対応するときだけ

R5秋 問28：更新可能なビューを作るSQL文		
	ビューの定義	更新
ア	**SELECT 商品番号, 商品名, 商品単価 … WHERE 商品単価 > 1000**	**できる（正解）**
イ	SELECT DISTINCT 商品番号	重複をまとめるので不可
ウ	SUM(受注数量) … GROUP BY	集計なので不可
エ	SELECT AVG(受注数量)	集計なので不可

R7秋 問28：ビューの列名は基の表と違う名前にできる（ア）。行の絞込み（WHERE）や和集合（UNION）でも定義できる。結合したビューは更新できないことがある。



仕組み	起動のしかた	過去問
ストアドプロシージャ	クライアントのCALL文。**通信量が減る**	R6春25（ア）
トリガー	データの変更をきっかけに**自動で**起動	R5春27エ（誤りの選択肢）

R6春25の誤答「実行計画の数・バッファ数・ディスクI/Oを減らす」は、DBMSの中の処理量の話。SQLの数は同じなので減らない。

第4章

トランザクション処理

ACID特性 ／ 専有ロックと共有ロック ／ デッドロック ／ MVCC ／ 隔離性水準 ／ 2相コミット ／ ロールバックとロールフォワード ／ チェックポイント ／ インデックス



ACIDは4つ。「可用性」「べき等」は仲間に見えて別のもの

A 原子性
全部か、何もしないか

C 一貫性
前後で矛盾がない

I 独立性
同時でも順番と同じ結果

D 耐久性
コミット後は消えない

R4秋 問30：ACIDに**含まれない**のは「可用性」（イ）。可用性はCAP定理のAやRASISの項目。

守る仕組み	どの性質か
ロールバック	原子性
制約（主キー・参照・検査）	一貫性
ロック・MVCC・隔離性水準	独立性
WAL（ログを先に書く）	耐久性

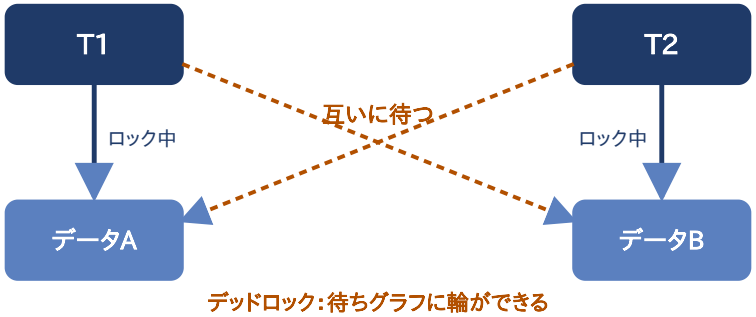
R5春 問28：べき等（idempotent）な操作の説明

	説明	正体
ア	**複数回実行しても1回と同じ結果**	**べき等**
イ	全部成功か、何も実行されないか	原子性
ウ	1つのノードへの挿入を他のノードでも実行	レプリケーション
エ	同時に実行しても順番に実行しても同じ	直列化可能性

「在庫を10にする」はべき等、「在庫を1減らす」はべき等でない。べき等なら通信の再送が安全。

🔒 読むなら共有ロック、書くなら専有ロック。待ち合いがデッドロック

かかっている＼かけたい	共有ロック（読む）	専有ロック（書く）
共有ロック	○ 同時にかけられる	× 待つ
専有ロック	× 待つ	× 待つ



用語	中身
ロック粒度	表・ページ・行。大きいと待ちが増え、小さいと管理の負荷が増える
2相ロッキング	ロックをかけ終えてから外し始める。直列化可能性を保つ
デッドロック対策	ロックの順番をそろえる。起きたら1つをロールバック（o08）
Wait-Die	古いほうは待ち、新しいほうがロールバック
Wound-Wait	古いほうが新しいほうをロールバックさせる
MVCC	データの**異なる版**を用意し、読みが書きを待たない（R7秋29イ）

2相**ロッキング**（同時実行制御）と2相**コミット**（分散トランザクション）、WALプロトコル（障害回復）を取り違えない。

⚠️ 隔離性水準を上げるほど、起きる現象が1つずつ消える

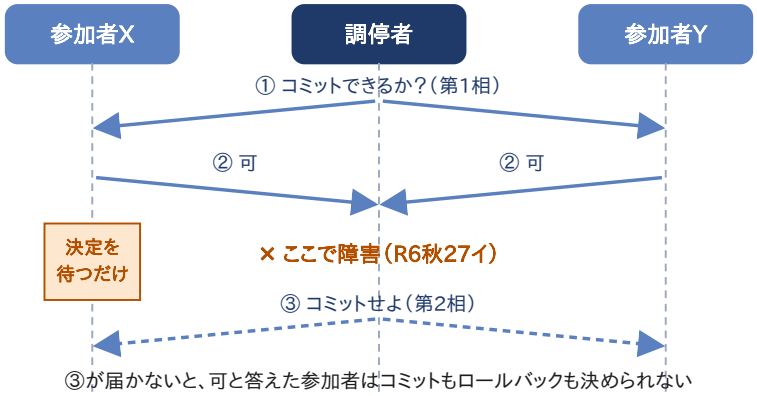
隔離性水準	ダーティ	ノンリピータブル	ファントム
READ UNCOMMITTED	起きる	起きる	起きる
READ COMMITTED	—	起きる	起きる
REPEATABLE READ	—	—	起きる
SERIALIZABLE	—	—	—

厳しくするほど待ちが増えて遅くなる。全部の現象を防ぐSERIALIZABLEは「直列に実行したのと同じ」。

現象	何が起きるか
ダーティリード	相手がまだコミットしていない値を読む。相手がロールバックすると幻の値（o07ア）
ノンリピータブルリード	同じ行を2回読むと**値**が変わっている
ファントムリード	同じ条件で2回検索すると**行の数**が変わっている
ロストアップデート	2人が同じデータを更新し、先の更新が上書きで消える

見分け方：「未コミット」ならダーティ、「値が変わる」ならノンリピータブル、「行が増える・減る」ならファントム、「更新が消える」ならロストアップデート。

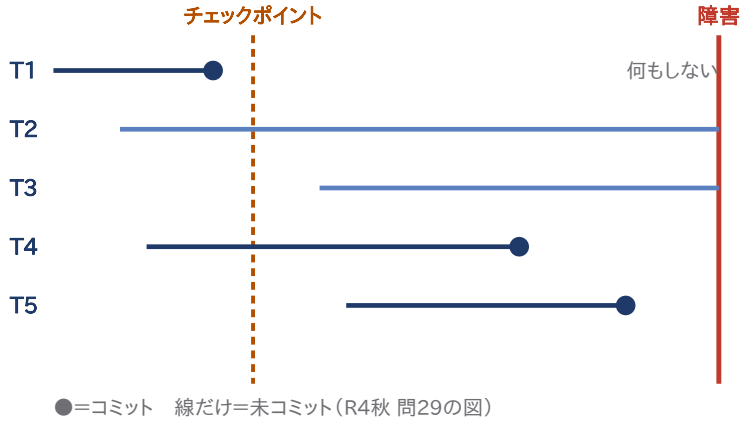
2相コミットは「聞いて、決めて、伝える」。伝える直前の障害が困る



R6秋 問27：参加者が判断できなくなる障害		
	障害が起きた時点	**判定**
ア	調停者が可否の問合せを送る直前	参加者は自分でロールバックできる
イ	**調停者が決定を送る直前**	**可と答えた後で待つしかない（正解）**
ウ	参加者が可否の応答を返す直前	調停者がタイムアウトでロールバック
エ	参加者が完了を返す直前	決定はもう届いている

ロックしない分散のやり方	中身
Sagaパターン	小さなトランザクションを順に実行し、失敗したら補償トランザクションで打ち消す
TCCパターン	仮押さえ（Try）→確定（Confirm）か取消し（Cancel）

⏳ チェックポイントの後にコミット→やり直す、未コミット→取り消す



●=コミット　線だけ=未コミット（R4秋 問29の図）

T	状態	処置
T1	チェックポイントより前にコミット	不要（書出し済み）
T2・T3	障害時に未コミット	ロールバック
T4・T5	**チェックポイントの後にコミット**	**ロールフォワード（ウ）**

判断はコミットの時点で。T4は開始がチェックポイントより前でも、コミットが後なのでロールフォワード。

R5秋 問30：未コミットでも**Readだけ**のトランザクション（T3・T4）は戻す更新がないのでロールバック不要。答えは「ロールフォワード T2・T5、ロールバック T6」（ア）。R6春 問27：チェックポイント後にコミットしたTはロールフォワード（エ）。

障害の種類で、使うログの情報が決まる

障害	回復の手順	使う情報
異常終了	そのトランザクションをロールバック	更新**前**情報
システム障害	コミット済みはロールフォワード、未コミットはロールバック（ウォームスタート）	両方＋チェックポイント
媒体障害	バックアップをリストア → ロールフォワード	更新**後**情報

R3秋 問27：障害回復の記述

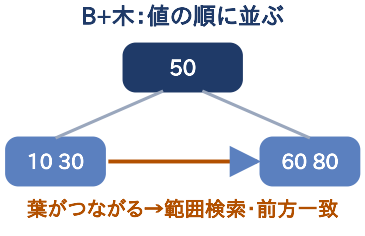
	記述	判定
ア	同時更新の矛盾防止にブロック化	防ぐのは排他制御
イ	**チェックポイント以前に終了したものは回復不要**	**正解**
ウ	媒体障害で更新前情報を使う	使うのは更新後情報
エ	異常終了で更新後情報を使う	使うのは更新前情報

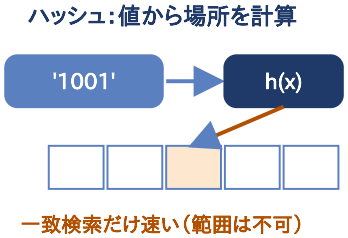
undo（取り消す）＝更新前情報　redo（やり直す）＝更新後情報
undo/redo方式 → 両方必要（R4春 問29 ア）

用語	中身
WALプロトコル	データより先にログをディスクへ。コミット済みを必ずやり直せる
チェックポイント	バッファをディスクに書き出す時点（R6秋31イ）
セーブポイント	トランザクションの途中の、部分ロールバック用の時点
シャドウページ法	新しいページに書き、対応表を切り替える。ログなし

> o09：差分バックアップの復元は「フル＋最後の差分」、増分は「フル＋その後の増分を全部」。

「等しい」ならハッシュ、「以上・今月・〜で始まる」ならB+木





R5秋 問26・R7秋 問27（同じ問題）：ハッシュのほうが適切な検索

	検索	向く索引
ア	売上金額が1万円以上	B+木（範囲）
イ	売上年月日が今月	B+木（範囲）
ウ	商品名が'DB'で始まる	B+木（前方一致）
エ	**利用者IDが'1001'**	**ハッシュ（正解）**

索引	向き・特徴
ビットマップ	性別のように値の種類が少ない列。AND・ORが速い
クラスタ化	行そのものをキー順に並べる。1表に1つ
カバリング	必要な列が全部索引にあり、表を読まない
転置	単語→文書。全文検索

R3春 問30：コストベースのオプティマイザに必要なもの

表の行数・値の分布などの統計情報（イ）

誤答のディメンジョンテーブル（スタースキーマの表）・待ちグラフ（デッドロックの検出）・ログファイル（障害回復）は、どれも別の章の用語。

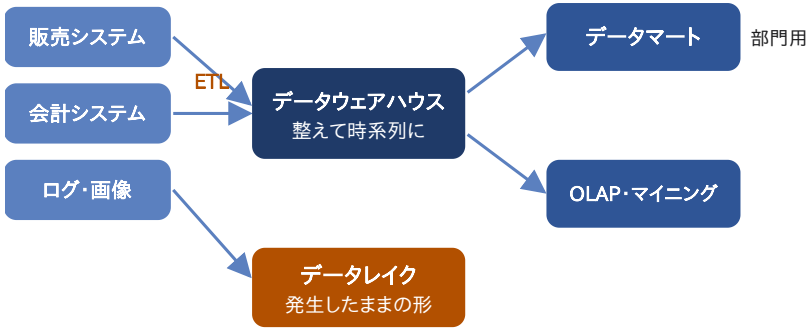
第5章

データベース応用

データウェアハウス・データマート・データレイク ／ ETLとELT ／ OLAPとデータマイニング ／ スタースキーマ ／ 分散の透過性 ／ CAP定理とBASE ／ ビッグデータとブロックチェーン



整えてためるのがデータウェアハウス、そのままためるのがデータレイク

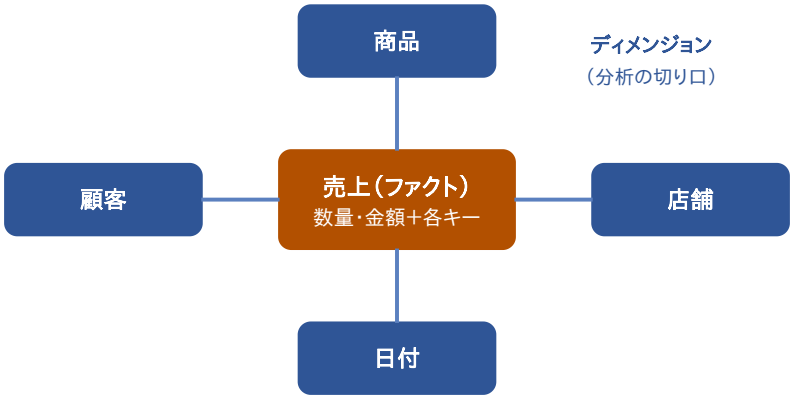


ETL＝取り出し→変換→書込み。ELT＝先に書き込んで、書込み先で変換（o11）。データクレンジング（誤り・表記揺れを直す）はTの段階。

R3春 問31（データレイク）とR4春 問30（データマイニング）の選択肢は同じ顔ぶれ	
説明	**正体**
隠れた規則や相関関係を見つけ出す	**データマイニング**（R4春30ア）
データウェアハウスから特定用途の分を取り出す	データマート
データウェアハウスやデータマートから多次元分析	OLAP
必要に応じて加工するため、発生したままの形で格納	**データレイク**（R3春31エ）
データを複数のサーバに複製し性能と可用性を向上	レプリケーション

日々の取引を処理するのがOLTP、ためたデータを分析するのがOLAP。

⭐ スタースキーマの中心は取引データのファクト。周りが切り口

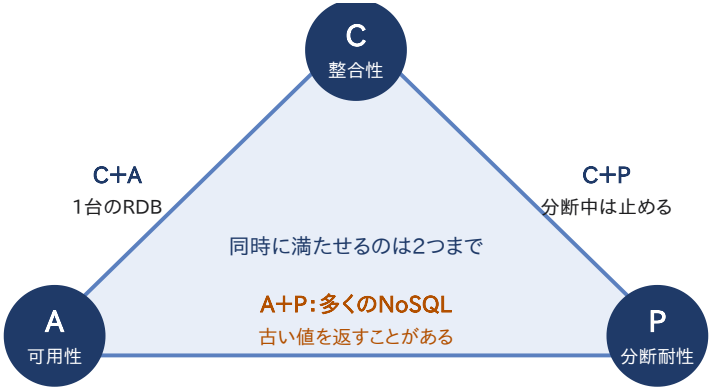


R6春 問28：分析対象のトランザクションデータを格納するのは**ファクトテーブル**（ウ）。ディメンジョンを正規化して枝分かれさせるとスノーフレークスキーマ。

OLAPの操作	中身
ドリルダウン	年→月→日と細かくする（o10エ）
ドリルアップ	日→月→年とまとめる
スライシング	ある次元を1つの値に固定して切り出す
ダイシング	集計の軸にする次元を入れ替える

表	入れるもの
ファクトテーブル	売上・数量などの事実。行数が最も多い
ディメンジョンテーブル	商品・顧客・日付などの切り口
サマリテーブル	よく使う集計結果をあらかじめ計算
ルックアップテーブル	コードと名称などの対応表

🌐 CAP定理は3つから2つ。AとPを選ぶと整合性を捨てて結果整合性に



R4春 問26：AとPをもつシステムは「可用性と分断耐性を満たすが整合性を満たさない」（ウ）。

R7秋 問26：BASE特性のNoSQL（更新を他の全ノードへ反映）

	記述	判定
ア	2相コミットで全ノードに反映	ACIDの厳しい整合性
イ	**障害がなければ、いつかは全ノードに反映**	**結果整合性（正解）**
ウ	同時の参照に全ノードが同じ結果	強い整合性
エ	分断されると受け付けない	CとPを選ぶ方式

BASE＝Basically Available・Soft state・Eventually consistent。ACID（酸）に対するBASE（塩基）。

スプリットブレイン：通信が切れて両方が主系として動き、データが食い違う状態。

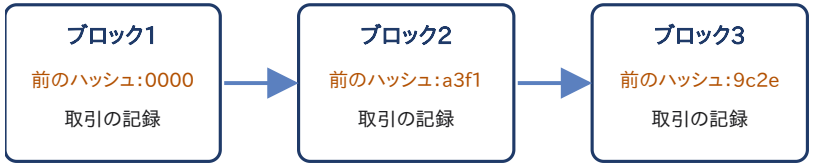
分散データベースの透過性は「何を意識しなくてよいか」で名前が決まる

R3秋 問30:“複製に対する透過性”の説明		
	プログラムが意識しなくてよいこと	**透過性**
ア	サーバごとにDBMSが異種	異種性
イ	1つの表が複数のサーバに分割されている	分割
ウ	表が別のサーバに移動された	移動(位置)
エ	**1つの表が複数のサーバに重複して存在**	**複製(正解)**

分散の方式	中身
レプリケーション	複製を別のサーバに作り、更新を反映し続ける
水平分割	行で分ける(東日本・西日本)
垂直分割	列で分ける(主キーは両方に残す)
シャーディング	水平分割したものを別々のサーバに置く
水平分散・垂直分散	同じ役割のサーバに分ける・役割ごとに分ける

ビッグデータの基盤	中身
Hadoop(HDFS+MapReduce)	ディスクに分散して格納・並列処理
Spark	メモリ上で分散処理。繰返しに強い
Hive・Presto	大量データにSQLのような問合せ

ブロックチェーンは、前のハッシュ値でつないだ記録を皆でもつ



1つを書き換えると、後ろのハッシュ値が全部合わなくなる

o12:改ざんが難しいのは「直前のブロックのハッシュ値を含んで鎖のようにつながり、多数の参加者が同じ記録をもつ」から(ア)。中央の管理者はいない。

用語	中身
コンセンサスアルゴリズム	どの記録を正しいとするか合意する手順
PoW	計算を最初に解いた者が追加。電力を大量に使う
PoS	保有量の多い者ほど追加しやすい
ファイナリティ	取引が確定し、覆らない状態
51%攻撃	計算能力の過半数で記録を書き換える
スマートコントラクト	条件を満たすと契約を自動で実行

データの形	例
構造化・半構造化・非構造化	表 ／ JSON・XML ／ 画像・音声・文書

📋 まとめ：表を1行ずつ追う問題と、説明を入れ替える問題の2種類

論点	覚えること	過去問
NoSQL	キー1つに値1つ＝キーバリュー、JSON＝ドキュメント、行ごとに列数自由＝ワイドカラム、ノードとリレーション＝グラフ	R3春28・R5春26・R7春24・R6春29
スキーマ	インデックスの定義は内部スキーマ。E-R・制約・正規化は概念スキーマ	R4春27
多重度	「1人以上」＝1..*、「何度も」＝*。外部キーは多の側	R3春29・R5春29・R6秋29・R7春25
正規化	主キーの一部で決まる→第2でない。キー以外どうし→第3でない	R3秋28・R4春28・R7春26
SQL	和・差・積の向き、UNIONで縦持ち、DISTINCT、NOT EXISTS、COALESCE(SUM,0)、PARTITION BY	R3秋26・29・R4秋27・28・R5秋29・R6春26・R6秋30
ビュー・手続	集約・DISTINCTのビューは更新不可。CALLはストアド、自動起動はトリガー	R5秋28・R7秋28・R5春27・R6春25
回復	チェックポイント後にコミット→ロールフォワード、未コミット→ロールバック、媒体障害→リストア＋更新後情報	R3秋27・R4春29・R4秋29・R5秋30・R6春27・R6秋31
同時実行	ACIDに可用性は入らない。MVCC＝版。2相コミットは決定を送る直前の障害が困る	R4秋30・R5春28・R7秋29・R6秋27
応用	データレイク＝そのまま、ファクト＝取引データ、AとP＝整合性を捨てる、BASE＝結果整合性	R3春31・R4春30・R6春28・R4春26・R7秋26・R3秋30